

第2章: Git と GitHub — コードを安全に管理する (完全版)

この章では、本アプリ「業務工数システム」のソースコードを **壊さず・失わず・安全に** 育てていくための道具、**Git (ギット)** と **GitHub (ギットハブ)** を、プログラミング完全初心者の目線で、**実際にこのプロジェクトで打つコマンドと、その実行結果の見え方** をひとつ残らず追いながら学びます。

プログラミングの世界で最も怖いのは「動いていたものが、いつのまにか動かなくなり、しかも元に戻せない」状態です。Git はこの恐怖から後任者を守る **タイムマシン** であり、**作業記録ノート** であり、**やり直しボタン** です。この章を読み終えると、あなたは「いつでも昨日の状態に戻せる」「自分が何を变えたか説明できる」「他人の変更と自分の変更を安全に合流させられる」ようになります。

⚠ この章には **絶対に守ってほしい運用ルール** があります。それは「**** commit (コミット)・push (プッシュ)** は、引き継ぎ担当者から明示的な指示があるまで実行しない**」というものです。理由は本文 § 16 で詳しく説明します。まずは「見るだけのコマンド (**status / diff / log**)」で Git に慣れてください。

この章は長いですが、**辞書のように使えること** を目指しています。実務で「あのコマンド何だっけ」と思ったら、ここに戻って該当箇所を引いてください。

この章で学ぶこと

- **バージョン管理とは何か** — なぜ「保存」だけでは足りないのか、Git が解決する具体的な困りごと
- **Git と GitHub の違い** — 「手元の道具」と「ネット上の倉庫」、混同しやすい2つをはっきり区別する
- **3つの場所 (作業ツリー／ステージング／リポジトリ)** — Git の心臓部を図で完全に理解する
- **見るコマンド** — `git status` / `git diff` / `git log` を実行結果つきで読む
- **記録するコマンド** — `git add` / `git commit`
(※指示があるまで実行しない、しくみだけ学ぶ)
- **同期するコマンド** — `git push` / `git pull` / `git fetch` (※同上)

- **ブランチと merge（マージ）** — 並行作業を安全にする枝分かれと合流
- **コンフリクト（衝突）の解決** — 実際の画面表示を1行ずつ追って手で直す
- **プルリクエスト（Pull Request）の流れ** — GitHub 上でのレビュー文化
- **.gitignore** — 本アプリの実ファイルを読み、**.env** を除外する重大な理由を理解する
- **トークン認証** — パスワードではなくトークンで GitHub に繋ぐ現代の作法
- **復旧コマンド** — `git restore` / `git reset` で「やってしまった」を元に戻す
- **1日の作業テンプレート** — 毎朝・作業中・帰る前にやることのチェックリスト

目次

0. 前提知識：なぜバージョン管理が必要なのか
 1. Git と GitHub の違い
 2. Git の3つの場所（最重要概念）
 3. このプロジェクトの Git の状態を見してみる
 4. `git status` — 今どうなっているか
 5. `git diff` — 何をどう変えたか
 6. `git add` — 記録の準備（ステージング）
 7. `git commit` — 記録を確定する
 8. `git log` — 過去の記録を読む
 9. リモートとの同期：push / pull / fetch
 10. ブランチ — 作業の枝分かれ
 11. merge（マージ） — 枝を合流させる
 12. コンフリクト（衝突）の解決
 13. プルリクエスト（Pull Request）の流れ
 14. `.gitignore` — 追跡しないファイルの指定
 15. トークン認証 — GitHub への安全なログイン
 16. 復旧コマンドと「やってしまった」対処集
 17. 1日の作業テンプレート
 18. トラブルシューティング
 19. 演習問題
 20. この章のまとめ
-

0. 前提知識：なぜバージョン管理が必要なのか

バージョン管理 (version control) とは？ ソフトウェアの「いつ・誰が・どこを・どう変えたか」を **記録し、いつでも好きな時点に戻せる** ように管理するしくみのこと。「バージョン」は「版」、つまり「ある時点のソースコードの姿」のことです。

0.1 「保存」だけでは足りない

ワープロや表計算なら、こまめに「上書き保存」していれば、たいてい安心です。ところがプログラムは、たった1文字の打ち間違いで全体が動かなくなり、しかも **どこを直したか自分でも分からなくなる** という事故が日常的に起きます。

初心者がよくやってしまう自己流の「バージョン管理」を見てみましょう。

```
業務工数システム_最新.zip
業務工数システム_最新_本物.zip
業務工数システム_最新_本物_これが正解.zip
業務工数システム_バックアップ_20260530.zip
業務工数システム_動いてたやつ.zip
```

⚠ これは **アンチパターン**（やってはいけない典型例）です。次のような困りごとがすべて起きます。

- どれが本当に最新か、ファイル名からは分からない
- 2つのzipの「どこが違うか」を人間の目で探すのは事実上不可能
- ディスクが zip だらけになる
- 「3日前のあの状態だけ戻したい」が叶わない（全部か無か）

0.2 Git が解決してくれること

Git を使うと、上の悩みがすべて消えます。Git はあなたの代わりに、次のことを自動でやってくれます。

困りごと	Git なしの世界	Git ありの世界
------	-----------	-----------

元に戻したい	zip を漁る	<code>git log</code> で時点を選んで一発で戻す
どこを変えたか知りたい	目で比較	<code>git diff</code> が変更行だけ色付き表示
誰が変えたか知りたい	分からない	<code>git log</code> に名前と日時が残る
複数人で同時に作業	zip の取り合い	ブランチで並行作業し、後で合流
なぜ変えたか思い出したい	記憶頼み	コミットメッセージに理由が残る

なぜ？ なぜ Git は変更を「行単位」で覚えられるのか Git は、保存のたびにファイル全体をコピーするのではなく、「前回と比べてどの行がどう変わったか（差分）」を計算して効率よく記録します。だから何百回コミットしても容量が爆発せず、しかも「3日前のこの1行だけ」を取り出せるのです。

0.3 本アプリでの実例 — コミットメッセージが語る歴史

本アプリ（業務工数システム）の実際の作業履歴を見てみましょう。これは作り話ではなく、このリポジトリに本当に残っている記録です。

```
30d45e7 人員一覧で異動・退社を後ろに変更
3de6e62 人員データ取り扱い修正
9156f15 休憩変更時の工数消去バグ修正
4d2469a 通知誤削除修正
43d02de 履歴に氏名追加
e414b68 工数合計追加
2816d8d 翌日チェック廃止
b5495cc 新アプリ移行に伴う大改修
```

左の `30d45e7` のような英数字は **コミットID（ハッシュ）**、右の日本語は **コミットメッセージ**（その変更の説明）です。これを読むだけで「このアプリがどう育ってきたか」が物語のように分かります。たとえば `9156f15 休憩変更時の工数消去バグ修正` は「休憩時間を変更したときに工数が消えてしまうバグを直した」回です。もし将来また同じあたりで不具合が出たら、この時点のコードを見れば「前回どう直したか」が分かります。

これが Git の真価です。 コードそのもののだけでなく、「**なぜそう変えたのか**」という意図が後任者に引き継がれます。だから本章では「良いコミットメッセージの書き方」も後で扱います。

1. Git と GitHub の違い

初心者が最初につまずくのが、**Git** と **GitHub** の混同です。名前が似ていますが、まったくの別物です。

1.1 ひとこと言うと

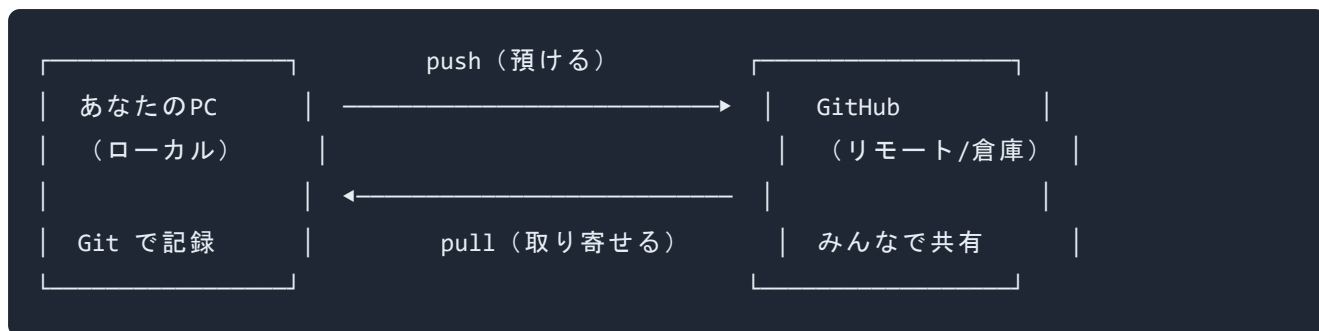
	Git	GitHub
正体	あなたのPCで動く ソフト（道具）	インターネット上の サービス（倉庫）
場所	手元（ローカル）	ネットの向こう（リモート）
作った人	Linus Torvalds（リーナス・トーバルズ）	GitHub社（現Microsoft）
必要か	必須 （これがないと始まらない）	あると便利（共有・バックアップ用）
たとえ	手元のノートとペン	みんなで使う図書館・貸金庫

用語: ローカル（local）とリモート（remote） 「ローカル」はあなたの目の前のPCの中。「リモート」はネットワークの向こう側にあるサーバー。本アプリのリモートは GitHub 上の <https://github.com/nomura-shokki/hozen-kosu-React.git> です（§9で確認します）。

1.2 たとえ話：研究ノートと図書館

イメージしやすいように、研究者にたとえます。

- **Git** は、あなたの机の上の「研究ノート」です。実験のたびに「いつ・何をやって・どうなったか」を書き込みます。これは **あなたのPCの中だけ** で完結します。ネットがなくても使えます。
- **GitHub** は、そのノートのコピーを預けておく「中央図書館」です。机が火事になってもノートは図書館に残ります。また、同僚も図書館に行けばあなたの最新ノートを読めますし、自分のノートを預けられます。



なぜ？ GitHub がなくても Git は使える Git は完全に手元で完結する道具です。極端な話、一度もネットに繋がず、自分一人で Git のメリット（タイムマシン・差分記録）を全部享受できます。GitHub は「そのノートを安全な場所にも置き、他人と共有するため」の追加のしぐみにすぎません。ただし本アプリはチーム開発でありバックアップも兼ねているので、GitHub と同期して使います。

1.3 GitHub の代わりになるサービス

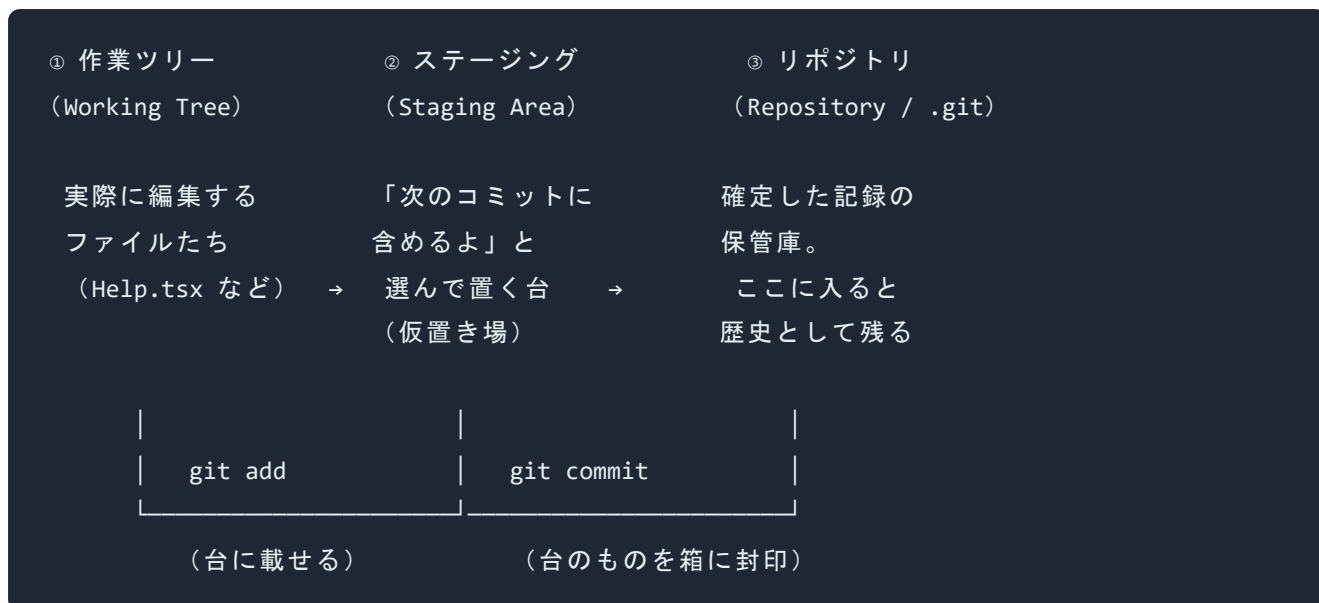
GitHub は最も有名ですが、同種のサービスは他にもあります（GitLab、Bitbucket など）。どれも「Git のリモート倉庫」という役割は同じです。本アプリは **GitHub** を使っているので、本章では GitHub を前提に説明します。

2. Git の3つの場所（最重要概念）

ここが本章で **いちばん大事** な節です。Git を理解できるかどうかは、この「3つの場所」を腹落ちさせられるかにかかっています。ゆっくり読んでください。

2.1 全体像の図

Git では、あなたのファイルは次の **3つの場所** を移動しながら記録されていきます。



ひとつずつ、たとえ話で説明します。

2.2 ① 作業ツリー (Working Tree)

作業ツリー (working tree : 作業ツリー) とは？ あなたが実際にエディタで開いて **編集している、今この瞬間のファイルたち** が置かれている場所。本アプリでいえば、`frontend/src/MainPage/Help.tsx` を開いて文字を打ち替えたら、その変更はまず「作業ツリー」に存在します。

たとえ: 料理でいう「まな板の上」。今まさに切ったり混ぜたりしている状態です。

この時点では、Git はまだ「変更を記録した」とは考えていません。「あなたが何かいじっているな」と気づいているだけです。

2.3 ② ステージング (Staging Area / Index)

ステージング (staging area : ステージングエリア、別名 index) とは？ 「次の記録 (コミット) に **含める変更** はこれですよ」と Git に教えるための **仮置き場 (中継台)**。 `git add` というコマンドで、作業ツリーの変更をこの台に載せます。

たとえ: 料理でいう「お皿に盛りつける直前の、トレーの上」。まな板の上のもの全部ではなく、「今回の一皿に使うものだけ」を選んでトレーに載せます。

なぜ？なぜわざわざ「仮置き場」を経由するのか 一度にたくさんのファイルを編集したとき、「この3ファイルは "バグ修正" として記録、残り2ファイルは "別の作業" として後で記録」のように、**変更を意味のあるまとまりに分けて記録できる** ようにするためです。トレー（ステージング）があるおかげで、「今回の記録に入れるもの」を自由を選べます。最初は難しく感じますが、「記録の下書き台」だと思えばOKです。

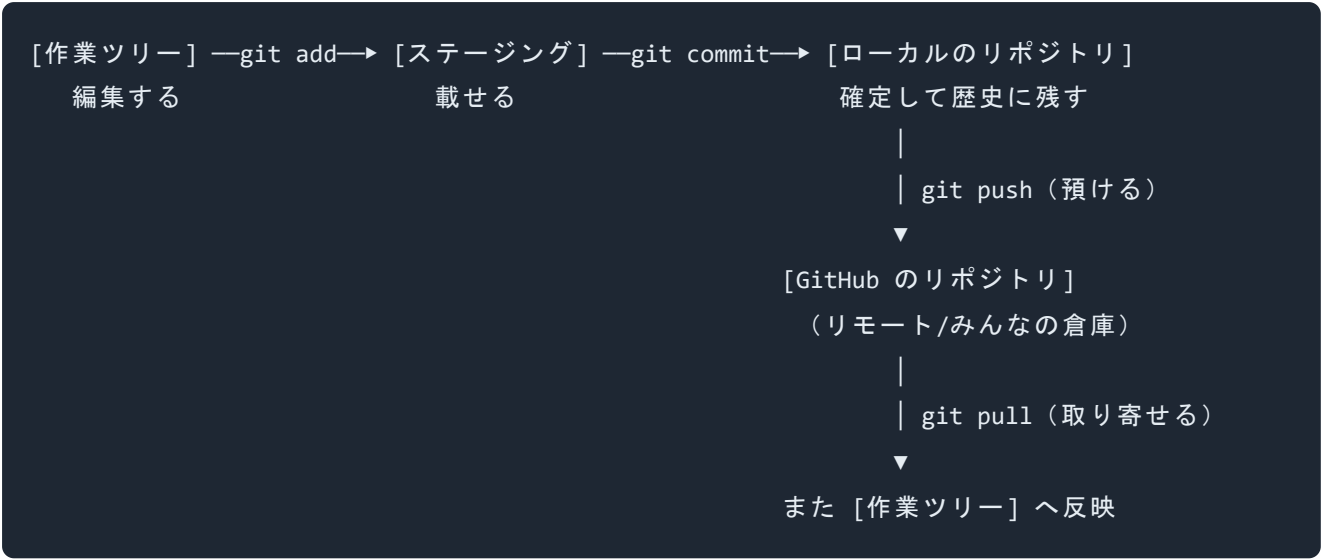
2.4 ③ リポジトリ (Repository)

リポジトリ (repository：レポジトリ、意味は「貯蔵庫」) とは？ 確定した変更の記録（コミット）が **歴史として永久に積み重なっていく保管庫**。あなたのPCの、プロジェクトフォルダ内にある `.git` という隠しフォルダの中身がこれです。 `git commit` というコマンドで、ステージングのトレーに載せたものを「一皿」として封印し、ここに収めます。

たとえ: 料理でいう「完成して出した皿の写真アルバム」。一度アルバムに貼った写真（コミット）は、後から「あの時の料理どうだったかな」と何度でも見返せますし、その時点に戻ることもできます。

用語: コミット (commit：意味は「確定する・約束する」) ステージングに載せた変更を「ひとつの記録単位」として確定する操作、またはその記録1個そのもの。コミット1個には「変更内容」「日時」「作者」「メッセージ (理由)」が含まれます。§ 0.3で見た `30d45e7` **人員一覧で異動・退社を後ろに変更** の1行が、コミット1個に対応します。

2.5 3つの場所と6つのコマンドの関係（まとめ図）



この図を頭に焼き付けてください。本章のコマンドはすべて、この図のどこかの矢印に対応しています。

コマンド	図のどの矢印か	ひとことで
<code>git add</code>	作業ツリー → ステージング	「次の記録に含める」と選ぶ
<code>git commit</code>	ステージング → ローカルリポジトリ	記録を確定する
<code>git push</code>	ローカル → GitHub	GitHub に預ける
<code>git pull</code>	GitHub → 作業ツリー	GitHub から取り寄せる
<code>git status</code>	(全体を見る)	今どの場所に何があるか確認
<code>git diff</code>	(作業ツリーを見る)	何をどう変えたか確認

3. このプロジェクトの Git の状態を試みる

ここからは、本アプリのフォルダで実際にコマンドを打ちます。**安全のため、まずは「見るだけ」のコマンドから始めます。** 見るだけのコマンドは、何も壊しません。何度打っても安全です。

前提: ターミナル（コマンドを打つ黒い画面）の開き方は **01** 章で扱いました。本アプリのフォルダ `hozen-kosu-React` を開いた状態で進めます。Windows では PowerShell、VS Code なら統合ターミナルです。

3.1 ここが Git 管理下かを確認する

▼ このコードがやること（先に日本語で）：今いるフォルダが Git で管理されているか（`.git` フォルダがあるか）をざっくり確認します。`git status` がエラーなく動けば管理下です。

```
# 今の状態を表示する。エラーが出なければ、ここは Git 管理下
git status
```

▼ こう表示されれば成功:

```
On branch master
...（変更ファイルの一覧など）
```

冒頭に `On branch master`（master ブランチにいます）と出れば、ここは正しく Git 管理下です。

⚠ もし `fatal: not a git repository`（致命的：ここは Git リポジトリではない）と出たら、フォルダを間違えています。`hozen-kosu-React` フォルダの中にいるか確認してください。

3.2 リモート（GitHub の場所）を確認する

▼ このコードがやること（先に日本語で）：このプロジェクトが「どの GitHub の倉庫」と繋がっているかを表示します。`-v`（verbose：詳しく）を付けると URL まで見えます。

```
# remote=リモート倉庫の一覧、-v=URLも表示
git remote -v
```

▼ こう表示されれば成功（本アプリの実際の出力）：

```
origin https://github.com/nomura-shokki/hozen-kosu-React.git (fetch)
origin https://github.com/nomura-shokki/hozen-kosu-React.git (push)
```

読み解きます。

- `origin` (オリジン) … リモート倉庫に付けられた **あだ名**。Git の慣習で、メインのリモートは `origin` と呼びます。「`origin` = GitHub の本アプリ倉庫」と覚えればOK。
- `https://github.com/nomura-shokki/hozen-kosu-React.git` … 実際の倉庫の住所 (URL)。
`nomura-shokki` というアカウントの `hozen-kosu-React` というリポジトリです。
- `(fetch)` … 取り寄せ用の住所。 `(push)` … 預け入れ用の住所。今回は同じです。

3.3 今いるブランチを確認する

▼ このコードがやること (先に日本語で) : 自分が今どの「ブランチ (作業の枝)」にいるか、どんなブランチが存在するかを一覧します。 `-a` (all) でリモートの枝も含めて全部表示します。ブランチの詳しい説明は § 10 です。

```
# branch=枝の一覧、-a=リモートも含めて全部
git branch -a
```

▼ こう表示されれば成功 (本アプリの実際の出力) :

```
* master
remotes/origin/HEAD -> origin/master
remotes/origin/master
```

読み解きます。

- `* master` … 先頭の `*` (アスタリスク) が「**今ここにいます**」の印。本アプリのメインブランチは `** master **` (マスター) です。
- `remotes/origin/master` … GitHub 側にある `master` ブランチ。
- `remotes/origin/HEAD -> origin/master` … リモートの既定の枝が `master` であることを示します。

用語: master (マスター) と main (メイン) Git のメインの枝は、歴史的に **master** と名付けられてきました。近年の GitHub では新規リポジトリの既定名が **main** に変わりましたが、**本アプリは master** です。本章では一貫して **master** と書きます。あなたが将来別のプロジェクトを触ると **main** のこともあるので、「メインの枝の名前」だと理解しておけば大丈夫です。

4. git status — 今どうなっているか

git status (ギット・ステータス) は、本章で **最も頻繁に打つコマンド** です。「今、3つの場所がどうなっているか」をいつでも教えてくれる、Git の健康診断です。困ったらまず **git status**。これが鉄則です。

4.1 基本

▼ **このコードがやること (先に日本語で)** : 今いるブランチ名、変更したけれどまだステージングしていないファイル、ステージング済みのファイル、Git がまだ追跡していない新規ファイルを、色分けして一覧します。

```
# status=状態。今の3つの場所の状態をまとめて表示
git status
```

4.2 本アプリでの実際の出力を読む

本アプリで **git status** を打つと、たとえば次のように出ます (実際の状態から抜粋・簡略化したもの)。

▼ **こう表示されます:**

```
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   frontend/src/MainPage/Help.tsx
        modified:   hozen_another/settings.py

no changes added to commit (use "git add" and/or "git commit -a")
```

1行ずつ読み解きます。

- **On branch master** … 今 **master** ブランチにいます (§ 3.3で見たとおり)。
- **Changes not staged for commit:** … 「**コミット用にステージングされていない変更**」の見出し。つまり「**作業ツリー** で変更したが、まだ **ステージング** に載せていない」ファイルたち。
- **(use "git add <file>..." to update ...)** … Git が親切に教えてくれるヒント。「これらをコミットに含めたいなら **git add ファイル名** してね」。
- **(use "git restore <file>..." to discard ...)** … 「変更を **捨てたい** なら **git restore ファイル名** してね」。**restore** は § 16で詳しく扱う「やり直しボタン」です。
- **modified: frontend/src/MainPage/Help.tsx** … **Help.tsx** (ヘルプ画面のファイル) を **変更 (modified) した** という意味。
- **modified: hozen_another/settings.py** … Django の設定ファイル **settings.py** も変更した。
- **no changes added to commit** … 「コミットに追加された変更はまだない」 = ステージングは空っぽ。

用語の早見表 (status に出てくる英単語) :

英語	意味	どの場所の話か
modified	既存ファイルを変更した	作業ツリー or ステージング
new file	新しく作って add 済み	ステージング
deleted	ファイルを削除した	作業ツリー or ステージング
Untracked files	Git がまだ追跡していない新規ファイル	作業ツリーのみ

Changes to be committed	コミットに含まれる予定 (add 済み)	ステージング
Changes not staged for commit	変更したがまだ add していない	作業ツリー

4.3 色の意味 (VS Code やターミナル)

多くの環境では色が付きます。

- **赤** … 作業ツリーにあるが、まだステージングされていない変更。
- **緑** … ステージング済み (`git add` した) 変更。次の `git commit` で記録される。

なぜ？ 赤と緑を見れば「あとどれを add すればいいか」が一目で分かる 「全部緑になった＝コミットに含めたいものが全部ステージングに載った」状態です。コミット前に `git status` で緑を確認するのが安全な習慣です。

4.4 短縮表示

たくさんファイルがある時は、`-s` (short: 短く) が便利です。

▼ **このコードがやること (先に日本語で)** : 状態を1ファイル1行のコンパクトな形式で表示します。

```
# -s=short。記号で簡潔に表示  
git status -s
```

▼ **こう表示されます (本アプリの章冒頭の状態に近い実例)** :

```
M frontend/src/MainPage/Help.tsx  
M hozen_another/settings.py  
M frontend/.env.production  
?? newfile.txt
```

行頭の記号: **M** = 変更 (modified)、**A** = 追加 (added)、**D** = 削除 (deleted)、**??** = 未追跡 (untracked)。左の桁がステージング、右の桁が作業ツリーの状態を表します。

5. git diff — 何をどう変えたか

`git status` は「どのファイルを変えたか」を教えてくださいますが、「**そのファイルの中身を、具体的にどう変えたか**」までは教えてくれません。それを見せてくれるのが `git diff` (ギット・ディフ、diff = difference = 差分) です。

5.1 作業ツリーの変更を見る (最頻出)

▼ **このコードがやること (先に日本語で)** : まだステージングしていない (赤い) 変更について、「変更前のどの行が、変更後どうなったか」を行単位で色付き表示します。

```
# ステージング前の変更を、行単位で表示
git diff
```

▼ **こう表示されます (例: Help.tsx の一部を変えた場合。※表示形式を示すための説明用の例)** :

```
diff --git a/frontend/src/MainPage/Help.tsx b/frontend/src/MainPage/Help.tsx
index 1a2b3c4..5d6e7f8 100644
--- a/frontend/src/MainPage/Help.tsx
+++ b/frontend/src/MainPage/Help.tsx
@@ -42,7 +42,7 @@
     <h2>ヘルプ</h2>
-    <p>このページでは使い方を説明します。</p>
+    <p>このページでは工数システムの使い方を説明します。</p>
     <ul>
```

1行ずつ読み解きます。これは初心者がいちばん戸惑う表示なので、丁寧にいきます。

- `diff --git a/... b/...` … 「**a** (変更前) と **b** (変更後) を比べます」の宣言行。

- `index 1a2b3c4..5d6e7f8 100644` … Git 内部のID。気にしなくてOK。
- `--- a/frontend/src/MainPage/Help.tsx` … `---` で始まる行は **変更前** の側。
- `+++ b/frontend/src/MainPage/Help.tsx` … `+++` で始まる行は **変更後** の側。
- `@@ -42,7 +42,7 @@` … 「**42行目から7行分** を表示しています」という位置案内（ハンク・ヘッダと呼びます）。
- `<h2>ヘルプ</h2>` … 行頭に記号なし = **変わっていない行**（前後の状況を見せるために表示）。
- `- <p>このページでは使い方を説明します。</p>` … 行頭が `-`（**マイナス、赤**） = **削除された行**（変更前の姿）。
- `+ <p>このページでは工数システムの使い方を説明します。</p>` … 行頭が `+`（**プラス、緑**） = **追加された行**（変更後の姿）。

覚え方: `-` は引いた（消した）行、`+` は足した（書いた）行。行を「書き換えた」場合は、古い行が `-` で消え、新しい行が `+` で足された、という2行ペアで表示されます。

5.2 ステージング済みの変更を見る

`git add` 済み（緑）の変更を見たいときは `--staged`（または `--cached`）を付けます。

▼ **このコードがやること（先に日本語で）:** ステージングに載せた変更だけを差分表示します。コミット直前に「これから記録される内容」を最終確認するのに使います。

```
# ステージング済み（add 後）の差分を表示
git diff --staged
```

なぜ？ コミット前に `git diff --staged` を打つと事故が減る 「記録するつもりがなかったデバッグ用の1行」や「うっかり消してしまった行」を、確定（commit）する前に発見できます。本章の運用では commit 自体を勝手に行いませんが、しくみとして知っておくと、後でレビューが速くなります。

5.3 特定ファイルだけ見る

▼ このコードがやること（先に日本語で）：ファイル名を後ろに付けると、そのファイルの差分だけを表示します。変更ファイルが多いときに便利です。

```
# このファイルの差分だけ見る
git diff frontend/src/MainPage/Help.tsx
```

5.4 diff の終了方法（初心者がハマる罠）

`git diff` や `git log` の出力が長いと、画面が ページャ（少しずつ表示するモード）に切り替わり、(END) や : が出てキーが効かなくなることがあります。

⚠ (END) や : が出て固まったように見えたら、q（quit の q）キーを押してください。これでプロンプト（コマンド待ち）に戻れます。Ctrl+C でも抜けられます。これは故障ではなく「閲覧モード」です。慌てないでください。

6. git add — 記録の準備（ステージング）

ここから先（`add` / `commit` / `push`）は 実際に状態を変えるコマンド です。

⚠ **本プロジェクトの運用ルール（再掲・最重要）**：`git add` までは比較的安全ですが、
** `commit` と `push` は、引き継ぎ担当者から明示の指示があるまで実行しないでください**
（理由は § 16.6）。この節と次節は「しくみを理解する」ことが目的です。手順は読んで頭に入
れつつ、実行は指示が出てからにしましょう。

6.1 基本

▼ このコードがやること（先に日本語で）：指定したファイルの「作業ツリーの変更」を、ステージング（次のコミットに含める仮置き場）に載せます。§ 2の図でいう「作業ツリー → ス

「ステージング」の矢印です。

```
# このファイルの変更をステージングに載せる
git add frontend/src/MainPage/Help.tsx
```

実行しても、画面には **何も表示されません**（成功時は無言）。確認は `git status` で行います。

▼ add 後に `git status` を打つとこう変わります:

```
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   frontend/src/MainPage/Help.tsx
```

`Changes not staged for commit`（赤）の見出しが `Changes to be committed`（緑）に変わりました。これが「ステージングに載った」証拠です。

6.2 複数ファイル・全部をまとめて

▼ このコードがやること（先に日本語で）：ファイルを並べれば複数まとめて、`.`（ドット）なら「変更した全ファイル」をステージングに載せます。

```
# 2つまとめて
git add frontend/src/MainPage/Help.tsx hozen_another/settings.py
# 変更した全部をまとめて
git add .
```

⚠ `git add .` は便利ですが、**要注意**。「意図せず変えてしまったファイル」や「コミットすべきでない一時ファイル」まで巻き込みます。`git add .` の **前に必ず `git status` を打って**、緑になる予定のファイルを確認する癖をつけてください。本アプリでは特に `frontend/build/` 配下の自動生成ファイルなどが紛れ込みやすいので注意します。

6.3 間違えて add したら (unstage)

ステージングから降ろしたい（作業ツリーには残したいが、今回の記録には含めたくない）場合：

▼ このコードがやること（先に日本語で）：ステージングに載せたファイルを「載せる前の状態（作業ツリーのみ）」に戻します。ファイルの中身は変わりません。

```
# ステージングから降ろす（中身は無傷）  
git restore --staged frontend/src/MainPage/Help.tsx
```

なぜ？ `--staged` を付けると安全 `git restore` は付ける引数で動きがガラッと変わります。`--staged` を付ける と「ステージングから降ろすだけ（中身そのまま）」で安全。`--staged` を付けない と「ファイルの中身を最後のコミット時点まで戻す（編集が消える）」破壊的操作になります。§ 16で詳述します。混同しないでください。

7. git commit — 記録を確定する

ステージングに載せた変更を「ひとつの記録（コミット）」として封印するのが `git commit` です。§ 2の図でいう「ステージング → ローカルリポジトリ」の矢印です。

⚠ 再掲: `commit` は 指示があるまで実行しない 運用です。ここはしくみと「良いメッセージの書き方」を学びます。

7.1 基本

▼ このコードがやること（先に日本語で）：ステージング済みの変更を1つの記録にまとめ、`-m` の後ろの文字列を「コミットメッセージ（変更理由）」として添えて確定します。

```
# -m=メッセージ。""の中が変更の説明
git commit -m "人員一覧で異動・退社を後ろに変更"
```

- `git commit` … 確定する命令。
- `-m` … message（メッセージ）の `m`。直後の `"..."` をコミットメッセージにします。
- `"人員一覧で異動・退社を後ろに変更"` … 何をしたかの説明。これは本アプリに実在するコミットメッセージ（§ 0.3）です。

▼ こう表示されれば成功:

```
[master 30d45e7] 人員一覧で異動・退社を後ろに変更
2 files changed, 14 insertions(+), 3 deletions(-)
```

- `[master 30d45e7]` … `master` ブランチに、ID `30d45e7` のコミットができた。
- `2 files changed, 14 insertions(+), 3 deletions(-)` … 2ファイル変更、14行追加、3行削除。

⚠ `-m` を付け忘れると、テキストエディタ（vim など）が開いて固まったように見えます。Vim が開いてしまったら、慌てず `Esc` キー → `:q!` と打って `Enter` で何もせず終了できます。初心者は必ず `-m "メッセージ"` を付けて、エディタが開かないようにしましょう。

7.2 良いコミットメッセージの書き方

コミットメッセージは「未来の自分と後任者への手紙」です。本アプリの実例から学びましょう。

良い例（本アプリの実際のメッセージ）:

人員一覧で異動・退社を後ろに変更	← 何を変えたか具体的
休憩変更時の工数消去バグ修正	← どんなバグを直したか分かる
履歴に氏名追加	← 機能追加の内容が明確
翌日チェック廃止	← 何を削除したか明確

これらは短いですが、「どの画面・機能を・どうしたか」が伝わります。本アプリは日本語のメッセージで統一されているので、後任者も日本語で書けばOKです。

⚠ 悪い例（やってはいけない）：

```
修正          ← 何を？どこを？が一切不明
aaa           ← 意味なし
いろいろ      ← 後で検索しても見つからない
とりあえず保存 ← Git は「保存」ではない。意味のまとまりで記録する
```

半年後の自分が `git log` を見て「これは何の修正だっけ？」とならないメッセージを心がけます。

なぜ？メッセージが歴史検索の鍵になる 「休憩まわりの不具合、前にも直したような…」というとき、コミット一覧から「休憩」という語を探すと、`9156f15 休憩変更時の工数消去バグ修正` が見つかります。メッセージが具体的だからこそ、過去の修正にたどり着けます。

7.3 コミットの粒度（どのくらいで1コミット？）

目安：「ひとつの意味のまとまり」で1コミット。たとえば「ヘルプ画面の文言修正」と「設定ファイルの変更」は **別々のコミット** にした方が、後で片方だけ戻せて便利です。本アプリの履歴も `履歴に氏名追加` `工数合計追加` のように、機能単位でコミットが分かれています。

8. git log — 過去の記録を読む

`git log`（ギット・ログ）は、これまでのコミット（歴史）を一覧するコマンドです。タイムマシンの「行き先一覧」です。これは **見るだけ** のコマンドなので、安心して何度でも打てます。

8.1 基本

▼ **このコードがやること（先に日本語で）：**コミットを新しい順に、ID・作者・日時・メッセージ付きで詳しく表示します。

```
# 全コミットを新しい順に詳しく表示
git log
```

▼ こう表示されます（本アプリの実際の履歴）：

```
commit 30d45e7c... (HEAD -> master, origin/master)
Author: yuya <oltotlo81@gmail.com>
Date: Sat May 30 ...
```

人員一覧で異動・退社を後ろに変更

```
commit 3de6e62...
Author: yuya <oltotlo81@gmail.com>
Date: ...
```

人員データ取り扱い修正

読み解きます。

- `commit 30d45e7c...` … コミットID（ハッシュ）。最初の7文字 `30d45e7` だけで指定できます。
- `(HEAD -> master, origin/master)` … `HEAD`（ヘッド）＝「今いる位置」。それが `master` を指し、`origin/master`（GitHub側）とも同じ位置、という意味。
- `Author: yuya <oltotlo81@gmail.com>` … このコミットを作った人。
- `Date:` … 作成日時。
- インデントされた `人員一覧で異動・退社を後ろに変更` … コミットメッセージ。

用語: HEAD（ヘッド） 「今、自分が見ている／立っているコミット位置」を指す矢印のようなもの。普段は最新コミットを指しています。タイムマシンでいう「現在地」。

8.2 1行表示（最頻出・おすすめ）

`git log` は1コミットが何行も使うので長くなりがち。普段は1行表示が便利です。

▼ このコードがやること（先に日本語で）：各コミットを「短いID+メッセージ」の1行に圧縮して一覧します。

```
# oneline=1行表示。ID+メッセージだけ  
git log --oneline
```

▼ こう表示されます（本アプリの実際の履歴）：

```
30d45e7 人員一覧で異動・退社を後ろに変更  
3de6e62 人員データ取り扱い修正  
9156f15 休憩変更時の工数消去バグ修正  
4d2469a 通知誤削除修正  
43d02de 履歴に氏名追加  
e414b68 工数合計追加
```

これは § 0.3 で見た一覧そのものです。普段の確認はこれで十分です。

8.3 件数を絞る・グラフ表示

▼ このコードがやること（先に日本語で）：最新の数件だけ見たり、ブランチの枝分かれをアスキーアートのグラフで見たりします。

```
# 最新5件だけ  
git log --oneline -5  
# ブランチの枝分かれをグラフで表示  
git log --oneline --graph --all
```

`--graph` を付けると、ブランチがどこで分かれてどこで合流したかが、線（`*`・`|`・`/`・`\`）で描かれます。§ 10～§ 11のブランチを学んだ後に見ると理解が深まります。

8.4 特定の時点の中身を見る

▼ このコードがやること（先に日本語で）：コミットIDを指定すると、その回で具体的に何行変わったか（diff）まで見られます。

```
# ID 9156f15（休憩バグ修正）の回の変更内容を表示
git show 9156f15
```

なぜ？ これが「タイムマシン」の威力 たとえば「休憩まわりがまた怪しい」となったとき、`git show 9156f15` で「前回どう直したか」を行単位で確認できます。コードの考古学（過去の意図を掘り起こす）に必須のコマンドです。

9. リモートとの同期：push / pull / fetch

ここまでは「手元（ローカル）」の話でした。ここからは GitHub（リモート）とのやり取りです。

⚠ 再掲: `push` は 指示があるまで実行しない 運用です（§16.6）。`pull` / `fetch` は「取り寄せ」なので比較的安全ですが、後述の注意を守ってください。

9.1 push — 手元の記録を GitHub に預ける

▼ このコードがやること（先に日本語で）：ローカルで確定したコミットを、GitHub（`origin`）の `master` ブランチに送って同期します。§2の図の「ローカル → GitHub」の矢印です。

```
# origin (GitHub) の master へ、手元のコミットを送る
git push origin master
```


▼ こう表示されれば成功:

```
Enumerating objects: 9, done.  
...  
To https://github.com/nomura-shokki/hozen-kosu-React.git  
3de6e62..30d45e7 master -> master
```

最後の `3de6e62..30d45e7 master -> master` が「この範囲のコミットを master に送った」の意味です。

⚠ **push は「公開・共有」の行為です。** 一度 push すると、他の人がそれを取り込みます。後から「やっぱり無しで」と取り消すのは大ごとです。だから本アプリでは **指示があるまで push しない** のです。

9.2 pull — GitHub の最新を手元に取り寄せる

▼ **このコードがやること（先に日本語で）** : GitHub 側にある最新のコミットを取得し、手元の作業ツリーに反映（合流）します。「取得（fetch）+ 合流（merge）」を一度に行います。

```
# GitHub の master の最新を取り寄せて合流  
git pull origin master
```

▼ こう表示されます:

```
Updating 3de6e62..30d45e7  
Fast-forward  
 frontend/src/MainPage/Help.tsx | 5 +++--  
 1 file changed, 3 insertions(+), 2 deletions(-)
```

Fast-forward（早送り）は「手元に変更がなかったなので、すんなり最新に追いつけた」という幸せな状態です。

⚠ **作業を始める前に必ず `git pull`。** 古い状態のまま編集すると、他人の変更とぶつかって § 12 のコンフリクトが起きやすくなります。§ 17 の「朝の儀式」に組み込みます。

9.3 fetch — 取得だけして、合流はしない

▼ **このコードがやること（先に日本語で）：** GitHub の最新を「取得するだけ」で、手元の作業ツリーにはまだ反映しません。中身を確認してから合流したい慎重派向け。

```
# 取得のみ。手元には自動で混ざらない
git fetch origin
```

pull と fetch の違い（重要）：

コマンド	取得	自動で合流	安全度
<code>git fetch</code>	する	しない （自分で確認できる）	高い
<code>git pull</code>	する	する （取得 + merge）	やや注意

`git pull` = `git fetch` + `git merge` です。慎重にいくなら `fetch` → `git log origin/master` で中身確認 → `git merge` の3手順に分けられます。

10. ブランチ — 作業の枝分かれ

ここから Git の真骨頂、**ブランチ（branch：枝）** です。

10.1 ブランチとは何か

ブランチ（branch：意味は「枝」）とは？ メインの歴史（`master`）から **枝分かれ** して、独立した作業を進められるようにするしくみ。枝の上で好きなだけ実験し、うまくいったら本流

(`master`) に **合流 (merge)** します。失敗したら枝ごと捨てればよく、本流は無傷です。

たとえ: 1冊のノート (`master`) に直接いきなり清書すると、失敗したとき台無しです。そこで「下書き用のコピー (ブランチ)」を作り、そこで試行錯誤し、完成したら本ノートに清書 (merge) する。失敗しても本ノートは綺麗なまま。

```
master:  A — B — C ————— F ← 本流 (いつでも動く状態を保つ)
              \               /
feature:      D — E ————— ← 枝 (ここで実験。完成したら C地点から F へ合流)
```

なぜ? なぜわざわざ枝を分けるのか ① 本流 (`master`) を常に「動く状態」に保てる。② 複数人が同時に別々の作業をしてもぶつからない。③ 「この機能だけやっぱり無し」が枝の削除で簡単にできる。本アプリのような業務システムでは、「本番で動いている `master` を壊さないこと」が最重要なので、ブランチを切って作業するのが安全です。

10.2 ブランチを作る・移動する

▼ **このコードがやること (先に日本語で):** `help-fix` という名前の新しい枝を作り、そこへ移動します。 `-c` (create) 付きの `switch` は「作って移動」を一度にやります。

```
# -c=create. help-fix という枝を作って移動
git switch -c help-fix
```

▼ **こう表示されれば成功:**

```
Switched to a new branch 'help-fix'
```

「新しいブランチ `help-fix` に切り替えました」。これ以降の編集・コミットは、`master` ではなく `help-fix` 上に積まれます。`master` は無傷のままです。

用語: switch と checkout 昔は `git checkout -b 名前` でブランチを作って移動していました。今は役割を分かりやすくした `git switch` (ブランチ移動専用) が推奨です。どちらも動きますが、本章では新しい `switch` を使います。

ブランチ名の付け方の例 (本アプリ向け) :

作業内容	おすすめのブランチ名
ヘルプ画面の文言修正	<code>help-text-fix</code>
工数合計機能の追加	<code>add-kosu-total</code>
休憩バグの修正	<code>fix-break-bug</code>

10.3 ブランチ間を行き来する

▼ **このコードがやること (先に日本語で)** : 既存のブランチへ移動します (`-c` なしの `switch`)。 `master` に戻りたいときに使います。

```
# master に戻る
git switch master
# help-fix に移動
git switch help-fix
```

▼ **こう表示されます:**

```
Switched to branch 'master'
```

⚠ **ブランチを移動すると、作業ツリーのファイルの中身も切り替わります。** `help-fix` で編集した内容は `master` に移ると見えなくなり (消えたわけではない)、 `help-fix` に戻ると再び現れます。「ファイルが消えた!」と慌てないこと。今どの枝にいるかは `git status` か `git branch` で確認できます。

11. merge（マージ） — 枝を合流させる

枝で完成した作業を本流（`master`）に取り込むのが **merge（マージ：合流）** です。

11.1 基本の流れ

merge は「**取り込みたい側（master）に移動してから、取り込む枝（help-fix）を指定する**」のが鉄則です。

▼ このコードがやること（先に日本語で）：まず本流 `master` に移動し、`help-fix` の変更を `master` に合流させます。

```
# ① 取り込み先（本流）に移動
git switch master
# ② help-fix の変更を master に取り込む
git merge help-fix
```

▼ こう表示されれば成功（Fast-forward の場合）：

```
Updating 3de6e62..a1b2c3d
Fast-forward
 frontend/src/MainPage/Help.tsx | 8 ++++----
 1 file changed, 4 insertions(+), 4 deletions(-)
```

用語: Fast-forward（ファストフォワード：早送り）マージ `master` が枝分かれ後に一切変わっていなかった場合、Git は「枝の先まで `master` のしおりを早送りするだけ」で済ませます。これが Fast-forward。最もシンプルで安全な merge です。

11.2 マージコミットができる場合

`master` 側も、枝を切った後に別の変更が進んでいた場合は、Git が両方を織り交ぜた「**マージコミット**」を自動で作ります。

▼ こう表示されます:

```
Merge made by the 'ort' strategy.  
frontend/src/MainPage/Help.tsx | 6 +++--  
1 file changed, 3 insertions(+), 3 deletions(-)
```

Merge made by the 'ort' strategy は「ort という方式で自動合流しました」の意味。**両方の変更が別々の場所なら、Git が自動でうまく混ぜてくれます。** 同じ場所を両方がいじっていると、次の §12 「コンフリクト」になります。

11.3 終わった枝を片付ける

▼ このコードがやること（先に日本語で）: master に取り込み終えた `help-fix` 枝を削除します。 `-d` (delete) は「ちゃんと merge 済みなら消す」安全版です。

```
# -d=delete。merge済みの枝を安全に削除  
git branch -d help-fix
```

⚠ `-d` (小文字) は「merge 済みでないと消さない」安全装置付き。 `-D` (大文字) は「未マージでも強制削除」で、未保存の作業が消えます。基本は **小文字の `-d`** を使ってください。

12. コンフリクト（衝突）の解決

コンフリクト (conflict : 衝突) とは? 2つのブランチ（または自分と他人）が **同じファイルの同じ行** を別々に書き換えたまま合流しようとして、Git が「どっちを正解にすればいいかわからない」と判断保留すること。Git は勝手に決めず、**人間に判断を委ねます。**

初心者はコンフリクトを怖がりますが、**正体さえ知れば必ず手で直せます。** ここで実際の画面を1行ずつ追います。

12.1 コンフリクトが起きた瞬間

`git merge` や `git pull` のときに、こう出ます。

▼ こう表示されたらコンフリクト発生:

```
Auto-merging frontend/src/MainPage/Help.tsx
CONFLICT (content): Merge conflict in frontend/src/MainPage/Help.tsx
Automatic merge failed; fix conflicts and then commit the result.
```

- `CONFLICT (content): Merge conflict in ...Help.tsx ... Help.tsx` の中身で衝突したよ。
- `Automatic merge failed; fix conflicts and then commit the result.` ... 自動合流は失敗。コンフリクトを直して、結果をコミットしてね。

12.2 どのファイルが衝突したか確認

▼ このコードがやること（先に日本語で）: 衝突して未解決のファイル一覧を確認します。

```
# 「both modified:」がコンフリクト中のファイル
git status
```

▼ こう表示されます:

```
Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   frontend/src/MainPage/Help.tsx
```

`both modified:` (両方が変更した) が、コンフリクト中のファイルです。

12.3 衝突マーカースを読む（ここが核心）

衝突したファイルを開くと、Git が次のような **目印（マーカー）** を埋め込んでいます。これは実際にファイルの中に書き込まれます。

▼ コンフリクト中の Help.tsx の中身（※形式を示すための説明用の例）：` ``

ヘルプ

<<<<<<< HEAD

このページでは使い方を説明します。

<p>工数システムの操作方法是こちらです。</p>

help-fix

1行ずつ意味を読み解きます。ここを理解できれば、もうコンフリクトは怖くありません。

- <<<<<<< HEAD … ここから下が「**今いるブランチ（master側、HEAD）の主張**」の始まり。

- `<p>このページでは使い方を説明します。</p>` … master 側が書いた内容。
- `=====` … 境界線。ここから下が「**取り込もうとした側（help-fix）の主張**」。
- `<p>工数システムの操作方法是こちらです。</p>` … help-fix 側が書いた内容。
- `>>>>>> help-fix` … help-fix 側の主張の終わり。

12.4 手で解決する（3つの選択肢）

あなたがやることは、「**マーカーを全部消して、正しい最終形だけを残す**」ことです。選択肢は3つ。

(A) master 側を採用する場合 → こう書き換える:

```
<h2>ヘルプ</h2>
  <p>このページでは使い方を説明します。</p>
  <ul>
```

(B) help-fix 側を採用する場合 → こう書き換える:

```
<h2>ヘルプ</h2>
  <p>工数システムの操作方法是こちらです。</p>
  <ul>
```

(C) 両方を活かして自分で書き直す場合 → こう書き換える:

```
<h2>ヘルプ</h2>
  <p>工数システムの操作方法（使い方）はこちらです。</p>
  <ul>
```

⚠ `<<<<<<<` , `=====` , `>>>>>>` の3種類のマーカー行は、1つ残らず削除してください。これらが1文字でも残っていると、コードが文法エラーになり、画面が真っ白になります。解決後に `git diff` で「マーカーが残っていないか」を必ず目視確認しましょう。

VS Code を使うと楽: VS Code はコンフリクト箇所に「Accept Current Change（master側を採用）」「Accept Incoming Change（help-fix側を採用）」「Accept Both Changes（両方）」というボタンを表示します。クリックするだけでマーカーごと処理してくれます。初心者にはこちらを強くおすすめします。

12.5 解決を確定する

直したら、Git に「解決したよ」と教え、合流を完了します。

▼ このコードがやること（先に日本語で）：解決したファイルを `add` で「解決済み」と記録し、`commit` で合流を完了します（merge 中の `commit` はメッセージ省略でもOK）。

```
# ① 「このファイルは解決した」と記録
git add frontend/src/MainPage/Help.tsx
# ② Unmerged が消えたか確認
git status
# ③ 合流を完了（※指示があれば実行）
git commit
```

▼ 解決後 `git status` でこう出れば成功:

```
All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)
```

「すべての衝突は解決済み。`git commit` で merge を締めくくってね」。

途中でやめたくなったら: `git merge --abort` と打てば、merge を始める前の状態に完全に巻き戻せます。「やっぱり今は merge しない」が安全にできる脱出ボタンです。コンフリクトで混乱したら、無理せず `--abort` で一旦戻り、落ち着いてから再挑戦しましょう。

13. プリクエスト（Pull Request）の流れ

プルクエスト（Pull Request、略して PR：プルリク）とは？「自分のブランチの変更を、本流（`master`）に取り込んでください」と GitHub 上で **お願い（リクエスト）** するしくみ。取り込む前に、他のメンバーが **レビュー（コードの確認）** できるのが最大の利点です。

ローカルの `git merge` と違い、PR は「**GitHub 上で、人の目を通してから合流する**」公式の手続きです。

13.1 PR の全体の流れ

- ① ブランチを作る (`git switch -c add-kosu-total`)
- ② そのブランチで作業し、commit する
- ③ `git push origin add-kosu-total` で GitHub に枝を送る
- ④ GitHub の画面で「Pull Request」を作成 (`add-kosu-total` → `master`)
- ⑤ レビューアが変更を確認し、コメント・承認 (Approve)
- ⑥ 「Merge」ボタンで `master` に合流
- ⑦ 枝を削除 (GitHub の Delete branch ボタン)
- ⑧ 手元で `git switch master && git pull` で最新に追いつく

⚠ ③ の `push` は **指示があるまで実行しない** 運用です。PR は「複数人で安全に合流する」ための仕組みなので、運用方針を引き継ぎ担当者に確認してから使います。

13.2 PR 画面で見る要素

GitHub の PR 画面には次の情報が並びます。

要素	意味
タイトル	何の変更か (例: 工数合計機能を追加)
Description (説明)	なぜ・どう変えたか・確認方法
Commits タブ	含まれるコミット一覧
Files changed タブ	変更されたファイルの diff (§ 5 の表示と同じ)
Conversation	レビューコメントのやり取り
Approve / Request changes	承認 / 修正依頼
Merge ボタン	押すと <code>master</code> に合流

なぜ？ なぜローカル merge ではなく PR を使うのか ① 第三者の目で **バグや危険なコード** を合流前に発見できる。② 「なぜこの変更をしたか」の議論が GitHub に記録として残る。③

master への直接の変更を防ぎ、本番システムの安全を守る。本アプリのような業務システムでは、この「合流前レビュー」が品質の生命線です。

13.3 PR の説明文の書き方（テンプレート例）

▼ PR 説明文のテンプレート（※説明用の例。本アプリ向けに調整可）：

```
## 何を変えたか
工数一覧に「合計」行を追加した。

## なぜ変えたか
利用者から「1日の合計をいちいち電卓で足している」との要望があったため。

## 確認方法
1. 工数ページを開く
2. 一覧最下部に「合計」が表示されることを確認
3. 値が各行の手計算と一致することを確認
```

本アプリのコミット `e414b68 工数合計追加` のような変更を PR にするなら、この粒度で書くと、レビューヤーが確認しやすくなります。

14. .gitignore — 追跡しないファイルの指定

`.gitignore`（ギット・イグノア、ignore＝無視する）とは？「Git に記録させたくない／追跡させたくない ファイルやフォルダ」を列挙する設定ファイル。ここに書いたものは、`git status` にも出てこなくなり、`git add .` でも巻き込まれません。

14.1 なぜ無視が必要なのか

世の中のファイルには「Git で管理すべきもの」と「すべきでないもの」があります。

管理すべき	管理すべきでない（無視する）
自分で書いたソースコード（ <code>.tsx</code> / <code>.py</code> ）	自動生成される <code>node_modules/</code> （巨大）

設定の ひな形	秘密情報 を含む .env
ドキュメント (.md)	ビルド成果物・キャッシュ (__pycache__ /)

14.2 本アプリのルート **.gitignore** を1行ずつ読む

これは本アプリに実在するファイル **/.gitignore** の **全文** です。1行ずつ意味を解説します。

▼ このファイルがやること（先に日本語で）：秘密情報ファイル **.env**、Python の自動生成キャッシュ、エディタ設定、巨大な依存フォルダ **node_modules** を、Git の追跡対象から外します。

```
.env                # ① 秘密情報（DBパスワード等）を含む環境設定ファイル。絶対に追
跡しない
**/__pycache__/*    # ② Pythonが自動生成するキャッシュフォルダ（全階層）。中身は再
生成される
*.py[cod]           # ③ コンパイル済みPython（.pyc/.pyo/.pyd）。自動生成物なので不
要
*$py.class          # ④ Jython等が作る .class ファイル。自動生成物
.vscode/            # ⑤ VS Codeの個人設定フォルダ。人によって違うので共有しない
.claude             # ⑥ Claude（AI支援ツール）の設定。個人環境用

node_modules/       # ⑦ npmが入れる大量の依存ライブラリ。巨大すぎ＆再生成可能
frontend/node_modules/ # ⑧ frontend配下の依存ライブラリも同様に無視
```

各行の補足：

- ① **.env**（ドットイーエヌブイ）… 環境変数（environment variables）を書くファイル。
CLAUDE.md にあるとおり、本アプリの **.env** には **SECRET_KEY**・**DB_PASSWORD**（データベースのパスワード）・**DB_HOST** などの **絶対に外に出してはいけない秘密** が入っています。詳細は §14.4。
- ② ****/__pycache__/*** … ****** は「どの階層でも」を意味するワイルドカード。Python が高速化のために自動生成するキャッシュ。消えても自動で再生成されるので記録不要。
- ③ ***.py[cod]** … ***** は「任意の文字列」、**[cod]** は「c か o か d のどれか1文字」。つまり **.pyc**・**.pyo**・**.pyd** をまとめて指定。すべて自動生成物。
- ⑤ **.vscode/** … エディタの個人設定。人によって好みが違うので共有しない（共有すると他人の設定を上書きして喧嘩になる）。

- ⑦⑧ `node_modules/` … `npm install` で入る何万ファイルもの依存ライブラリ。 `package.json` さえあれば誰でも再生成できるので、Git には入れない（入れるとリポジトリが数百MBに膨れる）。

14.3 本アプリの frontend `.gitignore` も読む

本アプリには `frontend/.gitignore` もあります。フロント専用の無視設定です。これも実在ファイルの全文です。

▼ このファイルがやること（先に日本語で）：フロント専用、依存フォルダ・テスト結果・OSやログの一時ファイル・本番用の環境設定を無視します。

```
# dependencies          # （コメント）依存ライブラリの見出し
/node_modules           # ① このフォルダ直下の node_modules を無視
# testing               # （コメント）テスト関連の見出し
/coverage               # ② テストのカバレッジ結果（自動生成）

# misc                  # （コメント）その他
.DS_Store               # ③ macOSが勝手に作る隠しファイル
npm-debug.log*          # ④ npmのデバッグログ（エラー時に出る）
yarn-debug.log*         # ⑤ yarnのデバッグログ
yarn-error.log*         # ⑥ yarnのエラーログ

.env.production         # ⑦ 本番用の環境設定（秘密を含みうる）。無視

# npm install           # （コメント）メモ。動作に影響なし
# npm run build         # （コメント）メモ
```

- # で始まる行は **コメント**（人間向けの注釈。Git は無視）。
- ⑦ `.env.production` … 本番（production）用の環境設定。秘密情報を含むため無視します。`git status`（章冒頭）で `M frontend/.env.production` のように出ることがありますが、これは過去に一度追跡対象になっていた経緯がある特殊ケースです。原則として `.env` 系は追跡しないのが鉄則です（§ 14.5の「すでに追跡してしまった場合」を参照）。

14.4 ⚠️ 最重要警告：`.env` を絶対に commit しない

⚠️ ⚠️ ⚠️ これは本章で最も重要な安全ルールです。`.env` には **データベースのパスワード**・`SECRET_KEY`（Django がセッションや暗号に使う秘密の鍵）が書かれています。これを

GitHub に push してしまうと、**世界中の誰でもパスワードを読めてしまい**、データベースが乗っ取られたり、データが盗まれたりします。

なぜ？ 一度 push した秘密は「消しても手遅れ」 GitHub は履歴をすべて残します。後から `.env` を削除してコミットしても、「過去のコミットを遡れば中身が見える」状態が残ります。さらに、誰かが既に clone（複製）していたら回収不能です。だから「**最初から1度も push しない**」ことだけが唯一の防御です。`.gitignore` に `.env` が入っているのは、この事故を物理的に防ぐためです。

もし誤って `.env` を add してしまったら:

```
git restore --staged .env # ステージングから降ろす（中身は無傷。§6.3）
git status                # .env が緑から消えたか確認
```

そして **絶対に commit/push せず**、引き継ぎ担当者に即報告してください。

14.5 すでに追跡してしまったファイルが無視に変える

`.gitignore` に書いても、**既に Git が追跡を始めてしまったファイル** には効きません。追跡を止めるには:

▼ **このコードがやること（先に日本語で）**: ファイルを「Git の追跡対象から外す」が、手元のファイル自体は消さない（`--cached`）。これで `.gitignore` が効くようになります。

```
# 追跡だけ外す。手元の .env ファイルは残る
git rm --cached .env
```

⚠ `--cached` を **付け忘れる** と、手元の `.env` ファイル **自体が削除** されます。必ず `--cached` を付けてください。これも commit を伴うので、本運用では実行前に指示を仰ぎましょう。

15. トークン認証 — GitHub への安全なログイン

`git push` や、プライベートリポジトリの `git pull` をすると、GitHub は「本当にあなたですか？」と認証を求めます。

15.1 なぜパスワードではなくトークンなのか

重要: GitHub は **2021年以降、HTTPS でのパスワード認証を廃止** しました。 `git push` のときにパスワードを入れても通りません。代わりに **トークン** を使います。

トークン (token: 意味は「引換券・合言葉」) とは? パスワードの代わりに使う、ランダムな長い文字列。正式には **Personal Access Token (パーソナル・アクセス・トークン、略して PAT)**。「この合言葉を持っている人=本人」とみなして GitHub が認証します。

なぜ? なぜトークンの方が安全なのか ① パスワードと違い **権限を細かく絞れる** (「読み取りだけ」「このリポジトリだけ」等)。② **有効期限を設定でき**、漏れても期限切れで無効になる。③ 漏れたら **そのトークンだけ即無効化** でき、パスワード (他サービスでも使い回しがち) を変える必要がない。

15.2 トークンの作り方 (手順)

GitHub の Web 画面で作ります。

- ① GitHub にログイン
- ② 右上アイコン → Settings (設定)
- ③ 左メニュー最下部 → Developer settings
- ④ Personal access tokens → Tokens (classic) → Generate new token
- ⑤ Note (メモ名): 例「kosu-react 用」
- ⑥ Expiration (有効期限): 例 90 days
- ⑦ Select scopes (権限): repo にチェック (リポジトリ操作に必要)
- ⑧ Generate token ボタン
- ⑨ 表示された「ghp_xxxxxxxx...」を必ずコピー (この画面を閉じると二度と見られない!)

⚠ **トークンは生成直後の一度しか表示されません。** 必ずその場でコピーし、パスワード管理ソフトなど安全な場所に保管してください。万一控え忘れたら、作り直す（古いものは削除）だけなので慌てなくて大丈夫です。

15.3 トークンの使い方

`git push` 時にユーザー名とパスワードを聞かれたら:

Username: あなたのGitHubユーザー名

Password: ここに「トークン」を貼り付ける（パスワードではない!）

覚え方: Password の欄に入れるのは **トークン** です。本物の GitHub ログインパスワードを入れても通りません。

15.4 毎回聞かれないようにする（資格情報の保存）

Windows では、一度入れたトークンを OS が安全に覚えてくれる設定があります。

▼ **このコードがやること（先に日本語で）:** Git に「認証情報を Windows の資格情報マネージャーに保存して」と設定し、次回から自動でトークンを使わせます。

```
# Windowsの資格情報マネージャーに保存
git config --global credential.helper manager
```

用語: --global（グローバル） 「このPC全体の設定として」の意味。 `--global` なしだと「このリポジトリだけ」の設定になります。認証ヘルパーは全リポジトリ共通でよいので `--global` を付けます。

⚠ **トークンが漏れたら即無効化。** GitHub の Settings → トークン一覧から、該当トークンの「Delete（削除）」を押せば、そのトークンは即座に使えなくなります。漏洩が疑われたら迷わ

ず削除→新規作成してください。

16. 復旧コマンドと「やってしまった」対処集

Git の最大の安心は「**たいていの失敗は元に戻せる**」ことです。ここでは「やってしまった！」を救う復旧コマンドを、危険度順に整理します。

16.1 編集を捨てて、最後のコミットの状態に戻す (restore)

▼ このコードがやること (先に日本語で) : 指定ファイルの **作業ツリーの編集を破棄** し、最後にコミットした時点の中身に戻します。「さっきの編集、なかったことにしたい」とき。

```
# このファイルの編集を捨てて元に戻す
git restore frontend/src/MainPage/Help.tsx
```

⚠ **これは破壊的操作です。** 捨てた編集は **基本的に戻りません** (まだコミットしていない変更なので、Git も覚えていない)。実行前に「本当に捨てていいか」を `git diff` で確認しましょう。

16.2 ステージングから降ろす (restore --staged)

§ 6.3 で見たもの。 `add` を取り消すだけで、中身は無傷の **安全な操作** です。

```
# add を取り消す (中身そのまま)
git restore --staged Help.tsx
```

16.3 直前のコミットメッセージを直す (amend)

▼ このコードがやること (先に日本語で) : 直前のコミットの **メッセージを書き直す** (まだ

push していない場合に限る)。

```
# 直前コミットのメッセージを上書き
git commit --amend -m "正しいメッセージ"
```

⚠ **--amend** は「直前コミットを作り直す」操作。すでに push したコミットに対して使うと、リモートと食い違って厄介になります。**push 前のローカルでのみ** 使ってください。本運用では commit/push を勝手にしないので、出番は限定的です。

16.4 コミットを取り消す (reset) — 危険度別

git reset (リセット) は、コミットを巻き戻す強力なコマンドです。3つのモードがあり、危険度が違います。

▼ このコードがやること (先に日本語で) : 直前のコミットを取り消す。 **--soft** は変更を残し、 **--mixed** (既定) はステージングだけ解除し、 **--hard** は **変更ごと完全に消す**。

```
# ① コミットだけ取消。変更はステージングに残る (安全)
git reset --soft HEAD~1
# ② コミット&add取消。変更は作業ツリーに残る (既定・安全)
git reset --mixed HEAD~1
# ③ コミットも変更も全部消す (超危険!)
git reset --hard HEAD~1
```

- **HEAD~1** (ヘッド・チルダ・いち) … 「今の位置 (HEAD) の1つ前」の意味。 **~2** なら2つ前。
- ① **--soft** … コミットは消すが、編集内容はステージングに残る。「コミットの粒度をやり直したい」とき。
- ② **--mixed** … コミットも add も取り消すが、編集は作業ツリーに残る。比較的安全。
- ③ **--hard** … **コミットも編集も跡形もなく消える**。取り返しがつかないことが多い。

⚠ ⚠ **git reset --hard** は本章で最も危険なコマンドです。作業ツリーの未保存の変更も、対象コミットも完全に消えます。使う前に必ず立ち止まり、可能なら引き継ぎ担当者に相談し

てください。初心者のうちは `--hard` は封印しておくのが安全です。

16.5 「コミット済みなら」だいたい救える (reflog)

救いの神 `git reflog` (レフログ) : Git は、HEAD が動いたすべての履歴 (reset も含む) を裏でこっそり記録しています。 `git reflog` を打つと、「reset で消したつもりのコミット」の ID が見つかり、 `git reset --hard そのID` で **復活** できることが多いです。「やらかした！」と思っても、**一度コミットしてさえいれば** 諦めるのはまだ早い。 `git reflog` を思い出してください。

```
# HEADが動いた全履歴を表示。消したつもりのコミットIDが見つかる
git reflog
```

16.6 ⚠️ 本プロジェクトの運用ルール (再々掲・最重要)

⚠️ **commit と push は、引き継ぎ担当者から明示の指示があるまで実行しないでください。**

なぜこのルールがあるのか:

1. **push は公開・共有の行為** (§ 9.1)。他のメンバーや本番環境に影響が及び、取り消しが困難。
2. **本アプリは業務システム**。 `master` が壊れると、実際の業務 (工数入力) が止まる恐れがある。
3. **.env 等の秘密情報の誤 push** (§ 14.4) は、取り返しのつかない情報漏洩につながる。
4. 初心者のうちは「何が安全で何が危険か」の判断が難しい。だから「見るだけ (status/diff/log)」で Git に慣れ、記録系 (add/commit/push) は **指示が出てから** が安全。

やってよいこと: `git status` / `git diff` / `git log` / `git branch` (一覧) / `git switch` (移動) など、**見る・移動するだけ** のコマンドは自由に練習して OK。 **指示を待つこと:** `git commit` / `git push` / `git reset --hard` / `git rm` など、**歴史や中身を確定・破壊する** コマンド。

17. 1日の作業テンプレート

最後に、毎日の Git の使い方を「儀式」としてテンプレ化します。これをルーティンにすれば、事故が激減します。

17.1 朝の儀式（作業を始める前）

```
# ① まず本流にいることを確認
git switch master
# ② 手元に変なゴミが残っていないか確認（クリーンが理想）
git status
# ③ GitHubの最新を取り寄せる（古い状態で始めない）
git pull origin master
# ④ 最近の変更をざっと把握
git log --oneline -5
```

なぜ朝に pull ? 昨日の自分や他の人の変更を取り込んでから始めることで、コンフリクト（§12）を未然に防げます。

17.2 作業中の儀式（こまめに）

```
# 今どのファイルに触ったか、いつでも確認
git status
# 何をどう変えたか、こまめに眺める
git diff
```

コツ: 「キリのいいところで `git status` と `git diff`」を口癖に。自分が何をしているか見失わなくなります。

17.3 帰る前の儀式（作業の区切り）

```
# 今日の変更を一覧で確認
git status
# 中身を最終確認
git diff
# ここから先（add/commit/push）は指示があれば実行：
# git add <意図したファイルだけ>
# git commit -m "わかりやすい日本語メッセージ"
# git push origin <ブランチ>
```

⚠ **add/commit/push の3行はコメントアウト（先頭に #）してあります。** §16.6 のルールどおり、これらは **指示が出てから** 実行します。それまでは **status** と **diff** で「今日やったこと」を自分の目で確認するところまでが日課です。

17.4 困ったときの合言葉

迷ったら、まず git status。Git は不安になったら必ず **git status** を打ちます。「今どの場所に何があるか」「次に何をすればいいか」のヒントが、必ずそこに書いてあります。本章のコマンドの9割は、**git status** を起点に組み立てられます。

18. トラブルシューティング

症状	原因	対処
<code>fatal: not a git repository</code>	Git 管理外のフォルダにいる	<code>hozen-kosu-React</code> フォルダに移動する
<code>git diff</code> で画面が固まり (END) が出る	ページャ表示モード	<code>q</code> キーで抜ける（故障ではない）
<code>git commit</code> でVimが開いて操作不能	<code>-m</code> を付け忘れた	<code>Esc</code> → <code>:q!</code> → <code>Enter</code> で抜ける。 次回は <code>-m "..."</code> を付ける

push でパスワードを入れても弾かれる	パスワード認証は廃止	Password 欄に トークン を入れる (§ 15)
CONFLICT が出た	同じ行を双方が変更	§ 12の手順でマーカーを消して解決。混乱したら git merge --abort
merge 後、画面が真っ白／文法エラー	<<<<<<< 等のマーカーが残っている	ファイル内を検索し、3種のマーカー行を全削除
git switch したらファイルが消えた	別ブランチに移動しただけ	元のブランチに git switch で戻れば再表示される
.env をうっかり add した	git add . で巻き込んだ	git restore --staged .env 、commit/push せず報告 (§ 14.4)
git reset --hard で作業が消えた	破壊的操作	git reflog で消えたコミットIDを探し復活を試みる (§ 16.5)
git pull でコンフリクト	リモートと手元が同じ行を変更	§ 12の手順で解決。朝の pull (§ 17.1) で予防
Your branch is ahead of 'origin/master' by N commits	手元にあるが未 push のコミット	状態説明にすぎない。push は指示があるまで保留
変更したのに git status に出ない	.gitignore で無視されている	§ 14で無視対象か確認。意図と違えば .gitignore を見直す

19. 演習問題

手を動かすと定着します。§ 16.6 のルールを守り、**commit / push / reset --hard** は行わないこと。「見る・移動する」練習だけで十分に力が付きます。

- 状態を見る:** 本アプリのフォルダで **git status** を打ち、今 **On branch master** と出ること、変更ファイルがあれば一覧されることを確認せよ。
- リモートを確認:** **git remote -v** を打ち、**origin** の URL が **https://github.com/nomura-shokki/hozen-kosu-React.git** であることを確認せよ。
- 歴史を読む:** **git log --oneline -10** を打ち、**30d45e7 人員一覧で異動・退社を後ろに変更** などのメッセージが並ぶことを確認せよ。気になるコミットを1つ選び、**git show <ID>** でその回の変更内容を眺めよ。

4. **差分を見る:** `Help.tsx` をエディタで1文字だけ書き換え（例: 文末に空白を足す）、`git diff` で `-`（赤）と `+`（緑）の行が出ることを確認せよ。確認したら `git restore frontend/src/MainPage/Help.tsx` で編集を捨てて元に戻せ。
5. **ブランチを練習:** `git switch -c practice-branch` で枝を作って移動し、`git branch` で `* practice-branch` と出ることを確認。 `git switch master` で戻り、`git branch -d practice-branch` で練習枝を片付けよ。
6. **.gitignore を読む:** `/.gitignore` を開き、`.env` が無視対象であることを確認。なぜ `.env` を絶対に push してはいけないのか、§ 14.4 を読まずに自分の言葉で説明できるか試せ。
7. **検索の練習:** `git log --oneline` の結果から「休憩」に関するコミット（`9156f15 休憩変更時の工数消去バグ修正`）を目で探し、そのIDで `git show` を打って、当時どこを直したか確認せよ。

20. この章のまとめ

- **バージョン管理**は「保存」の上位互換。Git は変更を **行単位** で記録し、**いつでも過去に戻せる** タイムマシン。`9156f15 休憩変更時の工数消去バグ修正` のようなメッセージが、コードの意図を後任者へ引き継ぐ。
- **Git は手元の道具（ローカル）、GitHub はネットの倉庫（リモート）**。本アプリのリモートは `origin = https://github.com/nomura-shokki/hozen-kosu-React.git`、メインブランチは `** master **`。
- Git の心臓は **3つの場所**：①作業ツリー（編集）→ `git add` → ②ステージング（記録の下書き台）→ `git commit` → ③リポジトリ（歴史の保管庫）→ `git push` → GitHub。
- **見るコマンド**（安全）：`git status`（今の状態）/ `git diff`（`-` 削除・`+` 追加の差分）/ `git log --oneline`（歴史一覧）。**困ったらまず `git status`。**
- **ブランチ**で本流を壊さず並行作業し、**merge**で合流。同じ行を双方が変えると **コンフリクト**。`<<<<<<< / ===== / >>>>>>>` のマーカーを **全部消して** 正解だけ残し、`add` → `commit` で解決。混乱したら `git merge --abort`。
- **PR（プルリクエスト）** は GitHub 上で「レビューしてから合流」する安全な手続き。業務システムの品質を守る要。
- `.gitignore` で `node_modules`・`__pycache__`・そして何より `.env`（**秘密情報**）を追跡から外す。`** .env` を1度でも push したら情報漏洩。最初から push しないことだけが防御。`**`
- GitHub 認証はパスワードではなく **トークン（PAT）**。Password 欄にトークンを貼る。漏れたら即無効化。
- 失敗しても **コミット済みなら `git reflog` でだいたい救える**。ただし `git reset --hard` は破壊的、初心者は封印推奨。

- ⚠ **本プロジェクトの鉄則**：`commit` / `push` は指示があるまで実行しない。見る・移動するコマンドで Git に慣れることから始める。
- **1日の儀式**：朝 `pull` で最新化 → 作業中こまめに `status` / `diff` → 帰る前に変更を `status` / `diff` で確認（add 以降は指示待ち）。

次は、画面の骨組みと見た目を作る言語、「[03_Web基礎_HTML_CSS.md](#)」へ進みます。